

useState() Hooks - Code and Real-World Examples

Modern React 18/19 | Senior Frontend Interview Guide

1. Five Progressive Examples

Example 1 - Basic: Counter

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <section>
      <p>Current count: {count}</p>
      <button onClick={() => setCount((previous) => previous - 1)}>Decrease</button>
      <button onClick={() => setCount((previous) => previous + 1)}>Increase</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </section>
  );
}
```

Explanation: count is the current state snapshot. The setter schedules the next render.

Expected output: Starts at 0; Increase and Decrease update the displayed count; Reset returns it to 0.

Interview point: Use functional updates when the next value depends on the previous value.

Performance: Cheap; the component and its descendants re-render.

Common mistake: Calling `setCount(count + 1)` repeatedly reads the same snapshot. Use `setCount(c => c + 1)`.

Example 2 - Practical Component: Controlled Customer Search

```
import { useState } from "react";

export default function CustomerSearch({ customers }) {
  const [query, setQuery] = useState("");
  const normalizedQuery = query.trim().toLowerCase();
  const visibleCustomers = customers.filter((customer) =>
    customer.name.toLowerCase().includes(normalizedQuery)
  );

  return (
    <section>
      <label htmlFor="customer-search">Search customers</label>
      <input id="customer-search" type="search" value={query}
        onChange={(event) => setQuery(event.target.value)} placeholder="Search by name" />
      <p>{visibleCustomers.length} customer(s) found</p>
      <ul>{visibleCustomers.map((customer) => <li key={customer.id}>{customer.name}</li>)}</ul>
    </section>
  );
}
```

Explanation: The input is controlled by query. Filtered customers are derived from props and query, not duplicated in state.

Expected output: Typing an shows customers whose names contain an.

Interview point: Do not store derived state unless it represents independently editable data.

Performance: Fine for small lists. For large lists, debounce remote search, virtualize results, or memoize expensive filtering after measurement.

Common mistake: Storing both query and `filteredCustomers` then manually synchronizing them.

Example 3 - Production Scenario: Quantity Selector

```
import { useState } from "react";

export default function ProductQuantitySelector({ initialQuantity = 1, availableStock, onQuantityChange }) {
  const [quantity, setQuantity] = useState(initialQuantity);

  function updateQuantity(nextQuantity) {
    const safeQuantity = Math.max(1, Math.min(nextQuantity, availableStock));
    setQuantity(safeQuantity);
    onQuantityChange?.(safeQuantity);
  }

  return (
    <section aria-label="Product quantity">
      <button type="button" onClick={() => updateQuantity(quantity - 1)} disabled={quantity <= 1}>-</button>
      <output aria-live="polite">{quantity}</output>
      <button type="button" onClick={() => updateQuantity(quantity + 1)} disabled={quantity >= availableStock}>+</button>
      <p>{availableStock} available</p>
    </section>
  );
}
```

Explanation: Local state owns the shopper's temporary selection. Stock is input data, not local state.

Expected output: Quantity remains between 1 and available stock.

Interview point: Keep state at the lowest owner; lift it only when another consumer needs coordinated control.

Performance: Avoid an API write for every rapid click; commit on Add to Cart or debounce intentionally.

Common mistake: Copying remote inventory into local state and allowing it to drift from the server.

Example 4 - Advanced: Profile Save Workflow

```
import { useState } from "react";

async function updateProfile(payload) {
  const response = await fetch("/api/profile", {
    method: "PATCH", headers: { "Content-Type": "application/json" }, body: JSON.stringify(payload),
  });
  if (!response.ok) throw new Error("Unable to update profile.");
  return response.json();
}

export default function DisplayNameForm({ initialDisplayName }) {
  const [displayName, setDisplayName] = useState(initialDisplayName);
  const [status, setStatus] = useState("idle");
  const [error, setError] = useState(null);

  async function handleSubmit(event) {
    event.preventDefault();
    const trimmedName = displayName.trim();
    if (!trimmedName) { setError("Display name is required."); return; }
    setStatus("saving"); setError(null);
    try { await updateProfile({ displayName: trimmedName }); setStatus("saved"); }
    catch (requestError) { setStatus("error"); setError(requestError.message); }
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="display-name">Display name</label>
      <input id="display-name" value={displayName}
        onChange={(event) => setDisplayName(event.target.value)} disabled={status === "saving"} />
      <button type="submit" disabled={status === "saving">{status === "saving" ? "Saving..." : "Save changes"}</button>
      {status === "saved" && <p role="status">Changes saved.</p>}
      {error && <p role="alert">{error}</p>}
    </form>
  );
}
```

Explanation: Editable form data, request lifecycle, and errors have distinct state responsibilities.

Expected output: The form disables while saving, displays success after completion, and shows request errors.

Interview point: For a contained form this is appropriate. Shared profile data and mutation caching should use a server-state layer.

Performance: Keep input state near the form to prevent app-wide keystroke re-renders.

Common mistake: Setting success before the request resolves or ignoring the error path.

Example 5 - Interview-Level: Expandable Analytics Table

```
import { memo, useCallback, useMemo, useState } from "react";

const AnalyticsRow = memo(function AnalyticsRow({ row, isExpanded, onToggle }) {
  const formattedRevenue = row.revenue.toLocaleString("en-US", {
    style: "currency",
    currency: "USD",
  });
  return (
    <>
      <tr>
        <td>
          <button
            type="button"
            onClick={() => onToggle(row.id)}
            aria-expanded={isExpanded}
          >
            {isExpanded ? "Hide" : "Show"}
          </button>
        </td>
        <td>{row.name}</td><td>{row.visits.toLocaleString()}</td><td>{row.conversionRate}%</td>
      </tr>
      {isExpanded && (
        <tr><td colspan="4">Revenue: {formattedRevenue}</td></tr>
      )}
    </>
  );
});

export default function AnalyticsTable({ rows }) {
  const [expandedIds, setExpandedIds] = useState(() => new Set());
  const sortedRows = useMemo(() => [...rows].sort((a, b) => b.visits - a.visits), [rows]);
  const handleToggle = useCallback((rowId) => {
    setExpandedIds((previousIds) => {
      const nextIds = new Set(previousIds);
      nextIds.has(rowId) ? nextIds.delete(rowId) : nextIds.add(rowId);
      return nextIds;
    });
  }, []);

  return (
    <table>
      <thead>
        <tr><th>Details</th><th>Campaign</th><th>Visits</th><th>Conversion</th></tr>
      </thead>
      <tbody>
        {sortedRows.map((row) => (
          <AnalyticsRow
            key={row.id}
            row={row}
            isExpanded={expandedIds.has(row.id)}
            onToggle={handleToggle}
          />
        ))}
      </tbody>
    </table>
  );
}
```

Explanation: State contains only expansion IDs; sorted rows are derived. A new Set is returned for every transition.

Expected output: Multiple campaign rows can expand independently.

Interview point: Set is mutable. Clone it before updating, never mutate the previous state instance.

Performance: memo and useCallback help only when props are stable. Use virtualization for very large tables.

Common mistake: `expandedIds.add(rowId); setExpandedIds(expandedIds)` mutates state and creates unpredictable results.

2. Real Production Scenarios

E-commerce: Product Variant Selection

Requirement: Select size, color, and quantity. Problem: Some variants are unavailable and selections affect price and media. Solution: Keep selected variant IDs and quantity locally; derive price and availability from catalog data. Why: This is local interaction state. Alternatives: URL state for shareable configuration, a global store for multi-route configuration. Trade-off: Local state is simple but resets on unmount; URL state supports sharing but adds synchronization work.

Banking: Transfer Form

Requirement: Manage recipient, amount, validation, confirmation, and submit status. Problem: Many fields have related, regulated transitions. Solution: Prefer `useReducer` for workflow state; use `useState` only for independent UI details. Why: Explicit actions improve auditability and testing. Alternatives: React Hook Form plus schema validation; a state machine. Trade-off: More boilerplate, but less ambiguous behavior.

CRM: Inline Contact Editing

Requirement: Edit contacts inside a dense table. Problem: Preserve drafts and handle failures without invalidating the canonical record. Solution: Keep `editingContactId` and draft state local; use server-state mutations for saving and cache invalidation. Why: Edit visibility is UI state while contact records are server state. Alternatives: Global store for a multi-page workflow. Trade-off: Row-local state is efficient but drafts can disappear when virtualized rows unmount.

Dashboard: Date Range and Filters

Requirement: Share filters across analytics widgets. Problem: Multiple consumers need the same state and users expect links to reproduce a view. Solution: Store filters in URL parameters or a scoped context. Why: This is navigable shared application state. Alternatives: Parent `useState` for a small one-page dashboard. Trade-off: URL state adds parsing but enables refresh, history, bookmarks, and supportable deep links.

Authentication: MFA Challenge

Requirement: Capture one-time code, show verify/resend states, and show errors. Problem: Input, timer, and network lifecycles differ. Solution: Use local state for code and temporary UI state; use mutation state for verification. Why: The typed code is transient; request state should be resilient and observable. Alternatives: A reducer or state machine for states such as expired, locked, and verified. Trade-off: `useState` is concise initially; a reducer is safer as workflow rules grow.

Search: Typeahead Suggestions

Requirement: Suggest results during typing. Problem: Rapid typing causes stale responses and excessive requests. Solution: Keep raw input in local state, debounce the query, cancel/ignore stale responses, and use cached server data for suggestions. Why: Input is UI state; suggestions are remote state. Alternatives: `useDeferredValue` for expensive local rendering or URL state for a results page. Trade-off: Debouncing reduces requests but intentionally delays results.

3. Performance Considerations

Re-render behavior

A state update schedules a render for the owner component and descendants. React reconciles the new tree and commits only necessary host updates. A re-render is not automatically expensive; broad state ownership, costly render work, unstable props, and large DOM collections are typical causes.

Memoization and optimization

Use `React.memo` for an expensive child that receives stable props. Use `useMemo` for expensive calculations, and `useCallback` only when function identity matters, such as memoized children. Measure first; indiscriminate memoization adds complexity.

Expensive work and bottlenecks

- Sorting/filtering large collections on each keystroke
- Large charts, rich text, expensive parsing, and long unvirtualized lists
- State lifted too high, causing broad render surfaces
- Large object updates for tiny changes

- Context values that change frequently
- Network requests started for every character

React 18+ automatically batches many updates and may skip work when next state is `Object.is` equal. For non-urgent expensive updates, use `startTransition`; for huge lists, virtualize rather than relying on memoization alone.

```
import { startTransition, useState } from "react";

function SearchPage() {
  const [query, setQuery] = useState("");
  const [filter, setFilter] = useState("");
  function handleChange(event) {
    const nextQuery = event.target.value;
    setQuery(nextQuery); // urgent input update
    startTransition(() => setFilter(nextQuery)); // non-urgent result update
  }
  return <input value={query} onChange={handleChange} />;
}
```

4. Comparison

Concept comparison

useState - **Purpose**: Local UI state; **Advantages**: Simple and explicit; **Disadvantages**: Related transitions can scatter; **Performance**: Re-renders owner and descendants; **When to choose**: Toggles, inputs, IDs

useReducer - **Purpose**: Related local transitions; **Advantages**: Named actions, testable logic; **Disadvantages**: More boilerplate; **Performance**: Similar render behavior; **When to choose**: Workflows and complex forms

useRef - **Purpose**: Persistent non-UI value; **Advantages**: No re-render; **Disadvantages**: UI cannot react to changes; **Performance**: Efficient for imperative values; **When to choose**: Timers, DOM nodes, previous values

useEffect - **Purpose**: External synchronization after commit; **Advantages**: Integrates subscriptions and APIs; **Disadvantages**: Misuse can add extra renders; **Performance**: Depends on effect work; **When to choose**: External systems, not derivation

useLayoutEffect - **Purpose**: DOM work before paint; **Advantages**: Avoids layout flicker; **Disadvantages**: Blocks paint; **Performance**: Higher perceived cost; **When to choose**: DOM measurement/correction

useMemo - **Purpose**: Cached calculation; **Advantages**: Avoids repeated expensive work; **Disadvantages**: Complexity, not a semantic guarantee; **Performance**: Helps only when calculation matters; **When to choose**: Proven expensive derivation

useCallback - **Purpose**: Cached function identity; **Advantages**: Helps memoized children; **Disadvantages**: Often unnecessary; **Performance**: Helps only with reference equality; **When to choose**: Stable callbacks required

5. Production Best Practices

- Store the minimum state required; derive totals, filters, and display flags.
- Keep state near its consumers and lift only to the lowest common owner.
- Separate UI state from server state; remote data needs caching, invalidation, retries, and cancellation.
- Use functional updates for any prior-state transition.
- Prefer IDs or normalized values over duplicated object snapshots.
- Treat arrays, objects, Maps, and Sets as immutable state values.
- Lazily initialize expensive values with `useState(() => createInitialValue())`.
- Use a reducer for action-oriented workflows and coordinated field transitions.
- Avoid effects that copy state to state; compute derived values during render.
- Measure before adding memoization; optimize expensive render paths, not every component.
- Put shareable filters and pagination in URL state where deep-linking matters.
- Model async UI explicitly: idle, loading, success, empty, and error.